

OMY-L100 소프트웨어 설치 및 활용 정리

대상 환경: **Ubuntu 22.04 Host + ROBOTIS Docker Container**

이 문서는 **Docker**를 사용하는 경우만 다룹니다.

Host PC에 ROS 2 Humble이 설치되어 있어도, OMY-L100 관련 실행은 ROBOTIS Docker 컨테이너 내부의 **Ubuntu 24.04 + ROS 2 Jazzy** 환경에서 수행하는 것을 기준으로 합니다.

0. Docker 전용 사용 방침

ROBOTIS 공식 OMY 문서는 개발용 사용자 PC에서 Docker 기반 환경을 사용할 수 있도록 안내하며, 컨테이너는 **Ubuntu 24.04 + ROS 2 Jazzy**로 동작하고 호스트 OS 버전은 컨테이너와 일치할 필요가 없다고 설명합니다.

따라서 Ubuntu 22.04에서 ROS 2 Humble을 사용 중이더라도, OMY-L100 관련 bringup, RViz, Gazebo, teleoperation 예제는 **전부 Docker 컨테이너 내부에서 실행**하는 방식으로 정리합니다.

OMY-L100은 OMY 라인업에서 **Leader unit for teleoperation** 역할을 하며, OMY-AI3M은 OMY-L100 leader와 OMY-F3M follower를 포함하는 leader-follower 시스템입니다.

참고:

- [ROBOTIS OMY Setup Guide](#)
- [ROBOTIS OMY Introduction](#)

1. 전체 구성 개요

1.1 하드웨어 관점

OMY-L100은 6-DOF leader 장치입니다. 공식 하드웨어 문서 기준으로 OMY-L100은 다음과 같은 구성을 사용합니다.

항목	내용
역할	Leader arm / teleoperation input device
자유도	6-DOF
Reach	560 mm
Weight	1.46 kg
Operating voltage	12 VDC
Host interface	U2D2 USB 2.0
Internal communication	TTL multidrop bus
Communication baudrate	4 Mbps

OMY 시스템 전체는 follower arm인 OMY-F3M과 leader device인 OMY-L100을 중심으로 imitation learning용 teleoperation system으로 설계되어 있습니다. L100은 사람의 손동작을 정밀한 robot command로 변환하는 경량 leader 장치로 볼 수 있습니다.

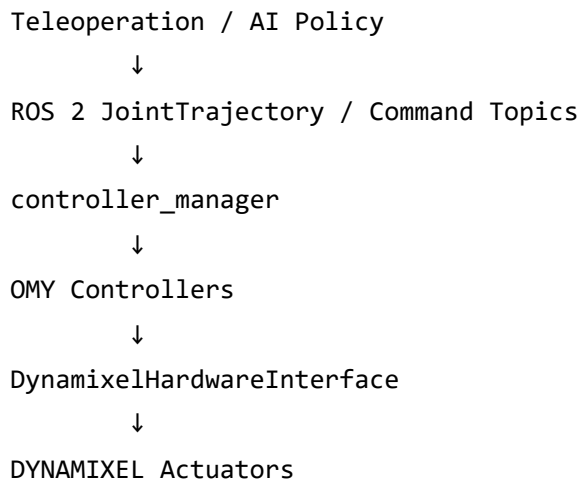
참고:

- [ROBOTIS OMY Hardware](#)

1.2 소프트웨어 관점

OMY 소프트웨어 스택은 ROS 2 Jazzy와 `ros2_control` 기반입니다. 공식 문서는 OMY가 ROS 2 Jazzy에서 동작하며, `ros2_control` 을 사용해 real-time joint-level control을 수행한다고 설명합니다.

제어 파이프라인은 대략 다음 구조입니다.



공식 문서도 teleoperation 또는 AI-generated command가 `ros2_control` 을 거쳐 hardware interface로 변환되고, DYNAMIXEL actuator로 실행되는 구조를 설명합니다.

참고:

- [ROBOTIS OMY Software](#)

2. Host PC 준비: Ubuntu 22.04

2.1 기본 패키지 설치

호스트 PC에서 최소한 다음을 설치합니다.

```
sudo apt update
sudo apt install -y git curl ca-certificates x11-xserver-utils
```

Docker Engine도 필요합니다. ROBOTIS setup guide는 OMY software setup prerequisites로 Docker Engine과 Git을 요구하며, Docker 설치 후 `docker` 그룹 등록, Docker service enable, `hello-world` 확인을 안내합니다.

```
sudo usermod -aG docker $USER
sudo systemctl enable --now docker
docker run hello-world
```

`sudo usermod -aG docker $USER` 실행 후에는 로그아웃/로그인 또는 재부팅이 필요합니다.

참고:

- [ROBOTIS OMY Setup Guide](#)

2.2 U2D2 USB 권한 설정

OMY-L100은 U2D2를 통해 PC와 USB로 연결됩니다. ROBOTIS U2D2 문서는 U2D2가 PC의 USB port에 연결되어 DYNAMIXEL과 통신하는 USB communication converter이며, TTL과 RS-485 connector를 지원한다고 설명합니다.

U2D2 자체는 DYNAMIXEL에 전원을 공급하지 않으므로, OMY-L100에는 별도 전원 공급이 필요합니다.

Ubuntu에서 `/dev/ttyUSB0` 같은 serial device에 접근하려면 사용자를 `dialout` 그룹에 넣습니다.

```
sudo usermod -aG dialout $USER
```

그다음 로그아웃/로그인 또는 재부팅합니다.

U2D2 연결 확인:

```
ls -l /dev/ttyUSB*  
dmesg | grep ttyUSB
```

일반적으로 OMY-L100이 하나만 연결되어 있으면 `/dev/ttyUSB0`로 보입니다. 여러 USB serial 장치가 있으면 `/dev/ttyUSB1`, `/dev/ttyUSB2` 일 수 있으므로 실제 포트를 확인해야 합니다.

참고:

- [ROBOTIS U2D2 e-Manual](#)

2.3 U2D2 latency timer 설정

OMY-L100은 4 Mbps 통신을 사용하므로 USB latency를 낮추는 것이 좋습니다. U2D2 e-Manual은 고속 통신 안정성을 위해 USB latency 값을 낮추는 것을 권장하며, Linux에서 `latency_timer`를 1 ms로 설정하는 명령을 제시합니다.

```
cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
```

재부팅 후에도 자동 적용하려면 udev rule을 추가합니다.

```
sudo tee /etc/udev/rules.d/99-u2d2-lowlatency.rules >/dev/null <<'EOF'
ACTION=="add", SUBSYSTEM=="usb-serial", KERNEL=="ttyUSB*", ATTR{latency_timer}="1"
EOF
```

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

참고:

- [ROBOTIS U2D2 e-Manual](#)

3. ROBOTIS Docker 환경 설치

3.1 작업 디렉터리 생성

```
mkdir -p ~/robotis_omy
cd ~/robotis_omy
```

3.2 공식 저장소 다운로드

ROBOTIS setup guide는 OMY에 필요한 저장소로 `open_manipulator` 와 `physical_ai_tools` 를 안내합니다.

- `open_manipulator` : ROS 2 control 기반 OMY arm 제어, teleoperation, low-level control 제공
- `physical_ai_tools` : data collection, training, inference, visualization을 포함하는 imitation learning toolkit

```
cd ~/robotis_omy
```

```
git clone -b jazzy https://github.com/ROBOTIS-GIT/open_manipulator.git
```

```
git clone -b jazzy --recursive https://github.com/ROBOTIS-GIT/physical_ai_tools.git
```

참고:

- [ROBOTIS OMY Setup Guide](#)
- [ROBOTIS-GIT/physical_ai_tools](#)

3.3 Docker 컨테이너 시작

```
cd ~/robotis_omy/open_manipulator  
./docker/container.sh start
```

컨테이너에 진입합니다.

```
./docker/container.sh enter
```

ROBOTIS setup guide의 container lifecycle도 ./docker/container.sh start , enter , stop 명령을 기준으로 설명합니다.

또한 컨테이너의 /workspace 는 호스트의 workspace 디렉터리와 volume mapping되어 보존되지만, 그 외 영역은 컨테이너 삭제 시 사라질 수 있다고 설명합니다.

참고:

- [ROBOTIS OMY Setup Guide](#)

3.4 ROS_DOMAIN_ID 설정

컨테이너 내부에서 실행합니다.

```
echo 'export ROS_DOMAIN_ID=30' >> ~/.bashrc  
source ~/.bashrc
```

ROBOTIS setup guide는 OMY 시스템에서 ROS 2 node 간 통신 충돌을 피하고 같은 network 내 통신을 맞추기 위해 `ROS_DOMAIN_ID=30` 을 설정하도록 안내합니다.

확인:

```
echo $ROS_DOMAIN_ID
```

참고:

- [ROBOTIS OMY Setup Guide](#)

3.5 GUI 사용을 위한 X11 접근 허용

Gazebo와 RViz는 GUI 프로그램이므로 호스트에서 X11 접근을 허용합니다.

호스트 터미널에서:

```
xhost +local:docker
```

공식 Gazebo guide는 Docker container에서 Gazebo GUI를 실행하기 전에 GUI access를 허용하기 위해 `xhost +` 를 실행하라고 설명합니다.

작업 종료 후 필요하면 접근을 닫습니다.

```
xhost -local:docker
```

참고:

- [ROBOTIS OMY Gazebo Guide](#)

3.6 Docker volume 및 device mapping 구조

ROBOTIS Docker 컨테이너는 OMY-L100 하드웨어 접근, ROS 2 workspace 보존, RViz/Gazebo GUI 실행을 위해 host resource를 컨테이너 내부로 mapping합니다. 일반적으로 사용자는 `docker run` 명령을 직접 작성하기보다는, ROBOTIS가 제공하는 `./docker/container.sh` 스크립트를 통해 컨테이너를 시작합니다.

핵심 mapping 구조는 다음과 같이 이해하면 됩니다.

Host path / resource	Container path / resource	목적
~/robotis_omy/open_manipulator/workspace 또는 ROBOTIS 스크립트가 지정한 workspace directory	/workspace	ROS workspace, build 결과, 사용자가 생성한 작업 파일 보존
/dev	/dev	U2D2, /dev/ttyUSB0 등 USB serial hardware device 접근
/tmp/.X11-unix	/tmp/.X11-unix	RViz, Gazebo 등 X11 GUI application 실행
\$DISPLAY	\$DISPLAY	Host X server로 GUI window forwarding
Host network	Host network	ROS 2 DDS discovery와 topic/service 통신 단순화
Docker container 내부 user/root filesystem	Container filesystem	패키지 설치, 임시 파일, 설정 파일 저장. 단, container 삭제 시 사라질 수 있음

중요한 점은 **보존해야 하는 파일은 /workspace 아래에 저장하는 것**입니다. 컨테이너 내부에서 /root , /home , /tmp 등에 저장한 파일은 컨테이너를 삭제하거나 재생성하면 사라질 수 있습니다. 반면 /workspace 는 host directory와 volume mapping되어 있으므로, 컨테이너를 재시작해도 유지됩니다.

예를 들어 컨테이너 안에서 직접 만든 launch file, calibration file, rosbag, dataset, log file 등을 보존하고 싶다면 다음과 같이 /workspace 아래에 저장하는 것이 안전합니다.

```
mkdir -p /workspace/my_omy_l100_tests
cd /workspace/my_omy_l100_tests
```

OMY-L100의 U2D2가 host에서 /dev/ttyUSB0 로 보인다면, 컨테이너 내부에서도 다음 명령으로 확인할 수 있어야 합니다.

```
ls -l /dev/ttyUSB*
```

컨테이너 내부에서 `/dev/ttyUSB0` 가 보이지 않는다면 다음 순서로 확인합니다.

```
# Host에서 장치 확인
ls -l /dev/ttyUSB*
dmesg | grep ttyUSB

# 컨테이너 재시작
cd ~/robotis_omy/open_manipulator
./docker/container.sh stop
./docker/container.sh start
./docker/container.sh enter

# 컨테이너 내부에서 다시 확인
ls -l /dev/ttyUSB*
```

RViz 또는 Gazebo GUI 실행에는 X11 socket과 display 환경 변수가 필요합니다. Host에서 다음 명령을 먼저 실행합니다.

```
xhost +local:docker
```

컨테이너 내부에서 display 환경 변수를 확인합니다.

```
echo $DISPLAY
```

ROBOTIS 스크립트가 실제로 어떤 volume과 device mapping option을 사용하는지 확인하고 싶다면 host에서 다음 명령으로 `container.sh` 내용을 확인할 수 있습니다.

```
cd ~/robotis_omy/open_manipulator
grep -n "workspace\|/dev\|X11\|DISPLAY\|network\|-v\|--volume" docker/container.sh
```

실전에서는 다음 원칙을 따르면 됩니다.

1. OMY-L100을 연결한 뒤 Docker container를 시작합니다.
2. 컨테이너 안에서 `/dev/ttyUSB0` 가 보이는지 확인합니다.
3. RViz/Gazebo를 실행하기 전에 host에서 `xhost +local:docker` 를 실행합니다.
4. 보존할 파일은 `/workspace` 아래에 저장합니다.
5. 컨테이너 내부의 `/root` , `/tmp` 등에 중요한 파일을 장기간 보관하지 않습니다.

4. 실제 OMY-L100 연결 전 점검

4.1 전원 및 USB 연결

OMY-L100 사용 전 다음을 확인합니다.

항목	확인 내용
전원	OMY-L100에 12 VDC 전원 공급
USB	U2D2와 사용자 PC 연결
Device file	/dev/ttyUSB0 또는 유사 포트 확인
권한	사용자 dialout 그룹 포함
Docker	/dev 가 컨테이너에 mapping되어야 함
통신	OMY-L100 baudrate 4 Mbps

ROBOTIS setup guide의 Docker volume configuration은 container가 hardware/system access를 위해 /dev:/dev 를 mapping한다고 설명합니다. 따라서 컨테이너 안에서도 /dev/ttyUSB0 장치에 접근할 수 있어야 합니다.

컨테이너 내부에서 확인:

```
ls -l /dev/ttyUSB*
```

참고:

- [ROBOTIS OMY Setup Guide](#)

4.2 DYNAMIXEL Wizard 2.0 또는 Workbench로 사전 확인

ROS bringup 전에 DYNAMIXEL ID, baudrate, firmware 상태를 점검하는 것이 좋습니다.

DYNAMIXEL SDK device setup 문서는 DYNAMIXEL을 PC에서 사용하려면 U2D2가 필요하며, 연결 설정 후 DYNAMIXEL Wizard 2.0으로 connection, motor ID, firmware 등을 확인하라고 안내합니다.

확인할 값:

Port : /dev/ttyUSB0
Baudrate : 4000000
Protocol : Dynamixel Protocol 2.0
Power : 12 VDC for OMY-L100

참고:

- [DYNAMIXEL SDK Device Setup](#)

5. OMY-L100 단독 동작 확인

5.1 Leader bringup 실행

컨테이너 내부에서 실행합니다.

```
cd ~/robotis_omy/open_manipulator  
./docker/container.sh enter
```

실제 OMY-L100 leader를 bringup합니다.

```
ros2 launch open_manipulator_bringup omy_l100_leader_ai.launch.py port_name:=/dev/ttyUSB0
```

omy_l100_leader_ai.launch.py 는 기본 hardware port를 /dev/ttyUSB0 로 선언하고, OMY-L100 URDF xacro를 사용합니다. 또한 leader namespace 아래에서 다음 node/controller를 실행하도록 구성되어 있습니다.

- ros2_control_node
- robot_state_publisher
- gravity_compensation_controller
- spring_actuator_controller
- joint_state_broadcaster
- joint_trajectory_command_broadcaster

참고:

- [omy_l100_leader_ai.launch.py](#)

5.2 동작 상태 확인

다른 터미널에서 컨테이너에 다시 진입합니다.

```
cd ~/robotis_omy/open_manipulator  
./docker/container.sh enter
```

토픽 확인:

```
ros2 topic list | grep leader
```

Joint state 확인:

```
ros2 topic echo /leader/joint_states
```

Controller 상태 확인:

```
ros2 control list_controllers --controller-manager /leader/controller_manager
```

정상이라면 OMY-L100을 손으로 움직였을 때 /leader/joint_states 의 position 값이 변해야 합니다.

OMY software guide도 /joint_states , controller 상태, RViz2를 주요 디버깅 수단으로 설명합니다.

참고:

- [ROBOTIS OMY Software](#)

6. 샘플·예제·테스트 프로그램 확보 및 구동

6.1 예제 프로그램은 어디에 있는가

Docker 전용 구성에서 예제와 launch 파일은 다음 저장소 안에 있습니다.

```
~/robotis_omy/open_manipulator  
~/robotis_omy/physical_ai_tools
```

주요 ROS 2 package:

```
open_manipulator_bringup
open_manipulator_description
open_manipulator_moveit_config
open_manipulator_gui
open_manipulator_teleop
open_manipulator_collision
```

공식 setup guide는 open_manipulator 가 OMY arm 제어, teleoperation, low-level control을 제공하고, physical_ai_tools 가 imitation learning의 data collection, training, inference, visualization을 담당한다고 설명합니다.

참고:

- [ROBOTIS OMY Setup Guide](#)

6.2 OMY-L100 leader 단독 테스트

```
ros2 launch open_manipulator_bringup omy_l100_leader_ai.launch.py port_name:=/dev/ttyUSB0
```

확인:

```
ros2 topic echo /leader/joint_states
ros2 control list_controllers --controller-manager /leader/controller_manager
```

이 테스트는 OMY-L100의 USB 통신, DYNAMIXEL 상태 읽기, gravity compensation 관련 controller 구동 여부를 확인하는 1차 테스트입니다.

6.3 OMY-L100 + OMY-F3M 실제 teleoperation 예제

OMY-L100의 대표 예제는 OMY-F3M follower를 조작하는 AI teleoperation입니다.

컨테이너 내부에서:

```
ros2 launch open_manipulator_bringup omy_ai.launch.py
```

공식 operation guide는 이 명령이 다음 절차를 순차적으로 실행한다고 설명합니다.

1. Follower 초기 위치 이동
2. Leader gravity compensation 시작
3. Leader-follower synchronization

또한 AI teleoperation은 OMY-F3M과 OMY-L100 모델 사이에서만 지원된다고 명시합니다.

실행 후 확인:

```
ros2 topic list | grep leader
ros2 topic list | grep joint
ros2 control list_controllers
```

참고:

- [ROBOTIS OMY Operation](#)

6.4 OMY-F3M follower 기본 bringup 예제

OMY-L100만 단독으로 확인한 뒤, 실제 follower arm까지 있다면 다음 예제를 실행합니다.

```
ros2 launch open_manipulator_bringup omy_f3m.launch.py
```

공식 operation guide는 Docker container 안에서 OMY package를 launch하는 기본 bringup 예제로 `omy_f3m.launch.py` 를 안내합니다.

참고:

- [ROBOTIS OMY Operation](#)

6.5 MoveIt 2 예제

OMY-F3M follower bringup 후 MoveIt 2를 실행합니다.

```
ros2 launch open_manipulator_moveit_config omy_f3m_moveit.launch.py
```

공식 operation guide는 MoveIt 2를 RViz에서 advanced motion planning용으로 실행하고, interactive marker를 움직인 뒤 Plan & Execute 로 robot arm motion을 실행하는 절차를 설명합니다.

참고:

- [ROBOTIS OMY Operation](#)

6.6 GUI 예제

OMY-F3M bringup과 MoveIt 2 실행 후 GUI를 실행합니다.

```
ros2 launch open_manipulator_gui omy_f3m_gui.launch.py
```

공식 GUI 예제는 다음 기능을 제공합니다.

- Start Timer
- robot status 확인
- init/home pose 이동
- gripper open/close
- joint space control
- task space control
- task constructor

참고:

- [ROBOTIS OMY Operation](#)

7. RViz에서 OMY-L100 가시화

7.1 실제 OMY-L100 상태를 RViz에서 보기

터미널 1, 컨테이너 내부:

```
ros2 launch open_manipulator_bringup omy_l100_leader_ai.launch.py port_name:=/dev/ttyUSB0
```

터미널 2, 컨테이너 내부:

```
rviz2
```

RViz에서 다음 display를 추가합니다.

```
TF
RobotModel
```

omy_l100_leader_ai.launch.py 는 robot_state_publisher 에 frame_prefix: leader_ 를 설정하고, 전체 node group을 leader namespace 안에서 실행합니다.

따라서 RViz의 Fixed Frame은 /tf_static 또는 RViz TF tree를 확인해 leader_ prefix가 붙은 base link 계열 frame으로 설정하는 것이 안전합니다.

TF 확인:

```
ros2 topic echo /tf_static
```

Joint state 확인:

```
ros2 topic echo /leader/joint_states
```

참고:

- [omy_l100_leader_ai.launch.py](#)

7.2 하드웨어 없이 OMY-L100 모델만 RViz에서 보기

실제 OMY-L100을 연결하지 않고 모델만 확인하려면 mock hardware를 사용합니다.

```
ros2 launch open_manipulator_bringup omy_l100_follower_ai.launch.py \
  use_mock_hardware:=true \
  start_rviz:=true
```

omy_l100_follower_ai.launch.py 는 다음 launch argument를 제공합니다.

- start_rviz
- use_mock_hardware
- port_name
- ros2_control_type

또한 OMY-L100 URDF xacro와 RViz config file을 사용하도록 구성되어 있습니다.

이 방법은 하드웨어 통신 문제와 무관하게 URDF, mesh, TF, RViz 설정을 먼저 확인할 때 유용합니다.

참고:

- [omy_l100_follower_ai.launch.py](#)

7.3 RViz 설정 파일 직접 실행

컨테이너 내부에서 다음처럼 RViz config를 직접 지정할 수 있습니다.

```
rviz2 -d $(ros2 pkg prefix open_manipulator_description)/share/open_manipulator_description/rvi:
```

RViz에서 RobotModel이 보이지 않으면 다음을 확인합니다.

```
ros2 topic list | grep robot_description
ros2 topic echo /tf_static
ros2 topic echo /leader/joint_states
```

8. Gazebo에서 OMY-L100 사용

8.1 중요한 전제

공식 Gazebo 문서의 기본 simulation launch는 OMY-F3M 기준입니다.

Gazebo guide는 Docker container 준비, GUI access 허용, container 진입 후 다음 명령으로 simulation을 실행하라고 안내합니다.

```
ros2 launch open_manipulator_bringup omy_f3m_gazebo.launch.py
```

따라서 Docker 전용 공식 경로에서 “Gazebo에서 OMY-L100 사용”은 보통 다음 의미로 해석하는 것이 안전합니다.

실제 OMY-L100 leader 입력

↓

/leader/joint_trajectory

↓

Gazebo 안의 OMY-F3M follower

즉, **Gazebo에 OMY-L100 자체를 독립 robot model로 띄우는 공식 기본 예제**라기보다, **실제 OMY-L100을 leader device로 사용해 Gazebo 안의 follower arm을 움직이는 예제**가 핵심입니다.

참고:

- [ROBOTIS OMY Gazebo Guide](#)

8.2 Gazebo GUI 준비

호스트 터미널:

```
xhost +local:docker
```

컨테이너 진입:

```
cd ~/robotis_omy/open_manipulator  
./docker/container.sh enter
```

8.3 기본 Gazebo simulation 실행

컨테이너 내부:

```
ros2 launch open_manipulator_bringup omy_f3m_gazebo.launch.py
```

공식 Gazebo guide는 이 명령으로 simulation environment를 실행하고, 이후 Operation page의 MoveIt/GUI 절차를 참고해 Gazebo 환경의 robot을 제어하라고 안내합니다.

참고:

- [ROBOTIS OMY Gazebo Guide](#)

8.4 실제 OMY-L100 leader로 Gazebo의 OMY-F3M follower 조작

터미널 1, 컨테이너 내부:

```
ros2 launch open_manipulator_bringup omy_f3m_follower_ai_gazebo.launch.py
```

터미널 2, 컨테이너 내부:

```
ros2 launch open_manipulator_bringup omy_l100_leader_ai.launch.py port_name:=/dev/ttyUSB0
```

omy_f3m_follower_ai_gazebo.launch.py 는 Gazebo simulation을 실행하고, OMY-F3M robot description을 spawn하며, arm_controller 의 trajectory topic을 /leader/joint_trajectory 로 remap하도록 구성되어 있습니다.

따라서 실제 OMY-L100 leader에서 생성되는 command를 Gazebo follower가 받아 움직이는 구조입니다.

확인:

```
ros2 topic list | grep leader
ros2 topic echo /leader/joint_states
ros2 topic list | grep trajectory
```

OMY-L100을 손으로 움직였을 때 Gazebo 안의 OMY-F3M이 따라 움직이면 정상입니다.

참고:

- [omy_f3m_follower_ai_gazebo.launch.py](#)

8.5 Gazebo + MoveIt 2

Gazebo simulation을 띄운 뒤 별도 컨테이너 터미널에서 MoveIt 2를 실행합니다.

```
ros2 launch open_manipulator_moveit_config omy_f3m_moveit.launch.py
```

RViz 안에서 interactive marker를 움직이고 Plan & Execute 를 눌러 trajectory planning을 확인합니다.

공식 operation guide는 MoveIt 2에서 planning group을 arm 으로 설정하고 predefined pose인 init 또는 home 을 사용할 수 있다고 설명합니다.

참고:

- [ROBOTIS OMY Operation](#)

8.6 Gazebo + GUI

Gazebo simulation과 MoveIt 2가 실행된 상태에서 GUI를 실행합니다.

```
ros2 launch open_manipulator_gui omy_f3m_gui.launch.py
```

GUI에서는 joint space, task space, saved pose 기반 task constructor 등을 확인할 수 있습니다.

참고:

- [ROBOTIS OMY Operation](#)

9. 자주 쓰는 점검 명령

모든 명령은 컨테이너 내부에서 실행합니다.

9.1 Node / topic 확인

```
ros2 node list
ros2 topic list
```

Leader 관련 topic:

```
ros2 topic list | grep leader
ros2 topic echo /leader/joint_states
```

Trajectory 관련 topic:

```
ros2 topic list | grep trajectory
```

9.2 Controller 확인

Leader controller manager:

```
ros2 control list_controllers --controller-manager /leader/controller_manager
```

일반 controller manager:

```
ros2 control list_controllers
```

9.3 TF 확인

```
ros2 topic echo /tf_static  
ros2 run tf2_tools view_frames
```

view_frames 실행 후 PDF가 생성되면 TF tree를 확인합니다.

9.4 USB 장치 확인

호스트 또는 컨테이너 내부:

```
ls -l /dev/ttyUSB*
```

호스트에서 kernel log 확인:

```
dmesg | grep ttyUSB
```

U2D2 latency 확인:

```
cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
```

10. 문제 해결 체크리스트

10.1 /dev/ttyUSB0 가 보이지 않는 경우

확인:

```
lsusb  
ls -l /dev/ttyUSB*  
dmesg | grep ttyUSB
```

조치:

```
sudo usermod -aG dialout $USER
```

로그아웃/로그인 또는 재부팅합니다.

10.2 컨테이너 안에서 /dev/ttyUSB0 가 보이지 않는 경우

컨테이너를 재시작합니다.

```
cd ~/robotis_omy/open_manipulator  
./docker/container.sh stop  
./docker/container.sh start  
./docker/container.sh enter
```

ROBOTIS setup guide 기준으로 컨테이너는 /dev 를 volume mapping하므로, 컨테이너 시작 시점에 장치가 연결되어 있는지 확인하는 것이 좋습니다.

참고:

- [ROBOTIS OMY Setup Guide](#)

10.3 RViz 또는 Gazebo GUI가 뜨지 않는 경우

호스트에서:

```
xhost +local:docker
```

컨테이너 내부에서:

```
echo $DISPLAY  
rviz2
```

Gazebo guide는 GUI access 허용 후 container에 진입해 Gazebo launch를 실행하는 순서를 안내합니다.

참고:

- [ROBOTIS OMY Gazebo Guide](#)

10.4 통신이 불안정한 경우

U2D2 latency를 확인합니다.

```
cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer  
echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
```

U2D2 문서는 안정적인 high baudrate communication을 위해 USB latency value를 낮게 설정하라고 설명합니다.

참고:

- [ROBOTIS U2D2 e-Manual](#)

11. 흥미로운 추가 활용: Physical AI / Imitation Learning

OMY-L100의 가장 흥미로운 활용은 단순 조이스틱이 아니라 **human demonstration** 수집 장치로 쓰는 것입니다.

OMY introduction 문서는 OMY가 human demonstration을 통한 skill acquisition, imitation learning, reinforcement learning을 지원하며, demonstration-based data collection부터 policy inference까지 end-to-end imitation learning pipeline을 제공한다고 설명합니다.

physical_ai_tools 저장소는 LeRobot과 ROS 2를 사용한 physical AI application 개발 interface를 제공한다고 설명하며, data collection, training, inference, visualization workflow와 연결됩니다.

대표 workflow:

```
OMY-L100으로 demonstration 수행
    ↓
OMY-F3M follower가 동작
    ↓
camera / joint state / action dataset 기록
    ↓
LeRobot 기반 policy training
    ↓
학습된 policy inference
    ↓
OMY-F3M에서 autonomous manipulation 수행
```

이 관점에서 OMY-L100은 “로봇을 움직이는 입력 장치”인 동시에, imitation learning dataset의 품질을 결정하는 핵심 teaching device입니다.

참고:

- [ROBOTIS OMY Introduction](#)
- [ROBOTIS-GIT/physical_ai_tools](#)

12. 권장 실행 순서

아래 순서대로 진행하면 가장 안정적으로 검증할 수 있습니다.

Step 1. Host 준비

```
sudo apt update
sudo apt install -y git curl ca-certificates x11-xserver-utils
sudo usermod -aG docker $USER
sudo usermod -aG dialout $USER
```

로그아웃/로그인 또는 재부팅.

Step 2. 저장소 다운로드

```
mkdir -p ~/robotis_omy
cd ~/robotis_omy

git clone -b jazzy https://github.com/ROBOTIS-GIT/open_manipulator.git
git clone -b jazzy --recursive https://github.com/ROBOTIS-GIT/physical_ai_tools.git
```

Step 3. Docker 시작

```
cd ~/robotis_omy/open_manipulator
./docker/container.sh start
./docker/container.sh enter
```

컨테이너 내부:

```
echo 'export ROS_DOMAIN_ID=30' >> ~/.bashrc
source ~/.bashrc
```

Step 4. U2D2 확인

호스트 또는 컨테이너 내부:

```
ls -l /dev/ttyUSB*
```

호스트에서 latency 설정:

```
echo 1 | sudo tee /sys/bus/usb-seria/devices/ttyUSB0/latency_timer
```

Step 5. OMY-L100 leader 단독 확인

컨테이너 내부:

```
ros2 launch open_manipulator_bringup omy_l100_leader_ai.launch.py port_name:=/dev/ttyUSB0
```

다른 컨테이너 터미널:

```
ros2 topic echo /leader/joint_states  
ros2 control list_controllers --controller-manager /leader/controller_manager
```

Step 6. RViz에서 OMY-L100 확인

하드웨어 사용:

```
rviz2
```

하드웨어 없이 mock model 확인:

```
ros2 launch open_manipulator_bringup omy_l100_follower_ai.launch.py \  
  use_mock_hardware:=true \  
  start_rviz:=true
```

Step 7. Gazebo simulation 실행

호스트:

```
xhost +local:docker
```

컨테이너:

```
ros2 launch open_manipulator_bringup omy_f3m_gazebo.launch.py
```

Step 8. 실제 OMY-L100으로 Gazebo follower 조작

터미널 1:

```
ros2 launch open_manipulator_bringup omy_f3m_follower_ai_gazebo.launch.py
```

터미널 2:

```
ros2 launch open_manipulator_bringup omy_l100_leader_ai.launch.py port_name:=/dev/ttyUSB0
```

Step 9. 실제 OMY-L100 + 실제 OMY-F3M teleoperation

```
ros2 launch open_manipulator_bringup omy_ai.launch.py
```

핵심 요약

- 이 문서는 **Docker 전용**입니다.
- Host Ubuntu 22.04의 ROS 2 Humble은 사용하지 않고, OMY 관련 실행은 컨테이너 내부 ROS 2 Jazzy에서 수행합니다.
- OMY-L100은 공식적으로 **teleoperation용 leader unit**입니다.
- 실제 하드웨어 확인의 1차 명령은 `omy_l100_leader_ai.launch.py` 입니다.
- RViz는 실제 leader bringup 후 `rviz2` 를 실행하거나, `use_mock_hardware:=true start_rviz:=true` 로 하드웨어 없이 확인할 수 있습니다.
- Gazebo의 기본 공식 simulation은 OMY-F3M 중심이며, OMY-L100은 실제 leader device로 사용해 Gazebo 안의 follower를 움직이는 구성이 핵심입니다.

- OMY-L100의 가장 중요한 확장 활용은 OMY-F3M과 결합한 teleoperation 및 imitation learning dataset 수집입니다.